



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE SISTEMAS DE INFORMACIÓN Y CONTROL

Programación Móvil

Investigación sobre conceptos de System Design

Profesor: Jorge Cisneros

Estudiante:

Guillén, Manuel Cédula: 8-1016-1618

Grupo: 1SF-241

Fecha de entrega: 10/06/2026

1. **Arquitectura Cliente-Servidor (Client-Server Architecture):** Modelo informático que separa las tareas entre los proveedores de recursos (servidores) y los demandantes (clientes). Un cliente envía una solicitud y el servidor devuelve una respuesta.
2. **Dirección IP (IP Address):** Etiqueta numérica única asignada a cada dispositivo conectado a una red informática que utiliza el Protocolo de Internet para la comunicación, permitiendo su identificación y localización.
3. **DNS (Domain Name System):** El "directorio telefónico" de Internet. Traduce los nombres de dominio legibles para los humanos en direcciones IP numéricas legibles para las máquinas.
4. **Proxy / Proxy Inverso (Reverse Proxy):** Un proxy actúa como intermediario entre un cliente e Internet. Un proxy inverso se sitúa frente a los servidores web y encauza las solicitudes de los clientes, mejorando la seguridad, el rendimiento y el balanceo de carga.
5. **Latencia (Latency):** El tiempo que tarda un paquete de datos en viajar desde su punto de origen hasta su destino. Una menor latencia significa una red más rápida y receptiva.
6. **HTTP/HTTPS:** Protocolo de Transferencia de Hipertexto. Es la base de la comunicación de datos en la web. HTTPS es la versión segura que encripta la comunicación usando TLS/SSL.
7. **APIs (Interfaces de Programación de Aplicaciones):** Conjunto de reglas que permiten que diferentes aplicaciones de software se comuniquen entre sí, definiendo los tipos de solicitudes que se pueden hacer y cómo se estructuran.
8. **REST API:** Estilo arquitectónico para APIs que utiliza peticiones HTTP estándar (GET, POST, PUT, DELETE) para acceder y manipular datos, caracterizado por ser sin estado (stateless) y orientado a recursos.
9. **GraphQL:** Lenguaje de consulta para APIs creado por Facebook que permite a los clientes solicitar exactamente los datos que necesitan y nada más, resolviendo problemas de sobre-búsqueda (over-fetching) o sub-búsqueda (under-fetching).
10. **Bases de Datos (Databases):** Sistemas organizados para la recopilación, almacenamiento, gestión y recuperación eficiente de datos estructurados o no estructurados.
11. **SQL vs NoSQL:** SQL se refiere a bases de datos relacionales estructuradas en tablas (ej. MySQL). NoSQL agrupa a bases de datos no relacionales, flexibles y orientadas a documentos, clave-valor o grafos (ej. MongoDB), ideales para escalar horizontalmente.
12. **Escalado Vertical (Vertical Scaling / Scale Up):** Aumentar la capacidad de un sistema añadiendo más potencia (CPU, RAM, almacenamiento) a un único servidor existente.
13. **Escalado Horizontal (Horizontal Scaling / Scale Out):** Mejorar el rendimiento y la capacidad de un sistema añadiendo más servidores (nodos) al grupo de recursos informáticos.
14. **Balanceadores de Carga (Load Balancers):** Dispositivos o software que distribuyen eficientemente el tráfico de red entrante entre un grupo de servidores de backend, evitando que un solo servidor se sobrecargue.
15. **Indexación (Indexing):** Técnica de optimización de bases de datos que crea estructuras de datos especiales para acelerar drásticamente la velocidad de las operaciones de búsqueda y recuperación.

16. **Replicación (Replication):** Proceso de copiar y mantener datos de la base de datos principal en múltiples servidores (réplicas) para mejorar la disponibilidad, tolerancia a fallos y rendimiento de lectura.
17. **Fragmentación (Sharding):** Técnica de escalado horizontal que consiste en dividir una base de datos enorme en partes más pequeñas y manejables llamadas "shards", las cuales se distribuyen en diferentes servidores.
18. **Partición Vertical (Vertical Partitioning):** Dividir una base de datos o tabla por columnas, moviendo ciertas columnas a una nueva tabla o servidor para aislar datos accedidos frecuentemente de los menos utilizados o datos grandes.
19. **Caché (Caching):** Técnica de almacenamiento temporal de copias de datos a los que se accede con frecuencia (en memoria RAM como Redis), para servirlos mucho más rápido en futuras solicitudes.
20. **Desnormalización (Denormalization):** Estrategia de optimización que añade intencionalmente datos redundantes a una base de datos para agilizar el rendimiento de lectura y evitar consultas complejas o uniones (joins) lentas.
21. **Teorema CAP (CAP Theorem):** Principio que establece que un sistema de datos distribuido solo puede garantizar simultáneamente dos de estas tres características: Consistencia (Consistency), Disponibilidad (Availability) y Tolerancia a Particiones (Partition tolerance).
22. **Almacenamiento Blob (Blob Storage):** Solución de almacenamiento en la nube para grandes cantidades de datos no estructurados, como archivos de texto o datos binarios (imágenes, vídeos, documentos), como AWS S3.
23. **CDN (Content Delivery Network):** Red de servidores distribuidos geográficamente que almacenan en caché contenido estático (imágenes, CSS, JS) para servirlo rápidamente a los usuarios desde el servidor más cercano a su ubicación.
24. **WebSockets:** Protocolo de comunicación que proporciona un canal bidireccional y persistente entre el cliente y el servidor, ideal para aplicaciones en tiempo real como chats o juegos.
25. **Webhooks:** Peticiones HTTP automáticas (generalmente POST) impulsadas por eventos. Permiten que una aplicación envíe datos en tiempo real a otra cuando sucede algo específico.
26. **Microservicios (Microservices):** Arquitectura donde una aplicación se estructura como una colección de servicios pequeños y autónomos que se comunican a través de APIs. Cada uno ejecuta un proceso único y se despliega independientemente.
27. **Colas de Mensajes (Message Queues):** Sistemas que permiten la comunicación asíncrona entre diferentes partes de una aplicación. Almacenan temporalmente los mensajes hasta que el servicio consumidor esté listo para procesarlos (ej. RabbitMQ, Kafka).
28. **Limitación de Tasa (Rate Limiting):** Técnica para controlar el tráfico limitando el número de peticiones que un usuario o cliente puede hacer a una API o servicio en un periodo de tiempo determinado, previniendo abusos y sobrecargas.
29. **Puertas de Enlace de API (API Gateways):** Servidor que actúa como único punto de entrada para un sistema de microservicios. Se encarga del enrutamiento de solicitudes, composición, limitación de tasa y autenticación.
30. **Idempotencia (Idempotency):** Propiedad de ciertas operaciones que garantiza que hacer múltiples peticiones idénticas tendrá el mismo efecto exacto que hacer una sola petición, previniendo resultados duplicados no deseados en la red.